3 — Dimensionality reduction

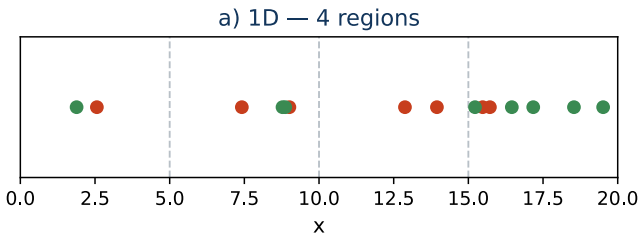
Machine Learning (MPA-MLR)

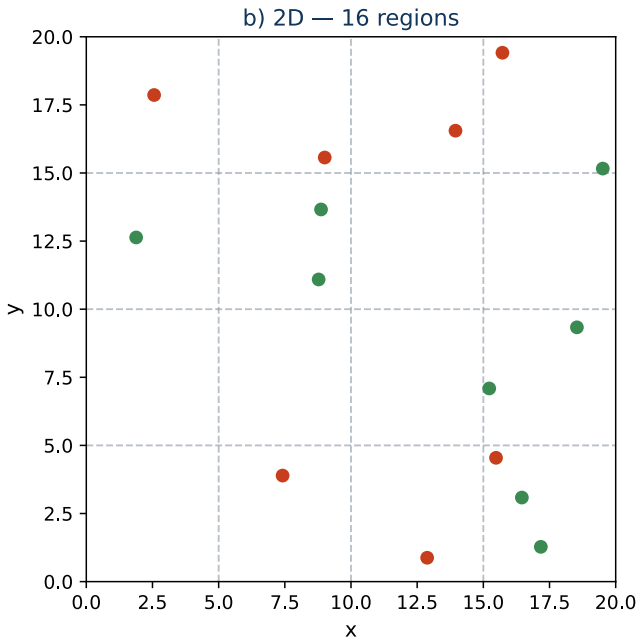
Tomáš Vičar

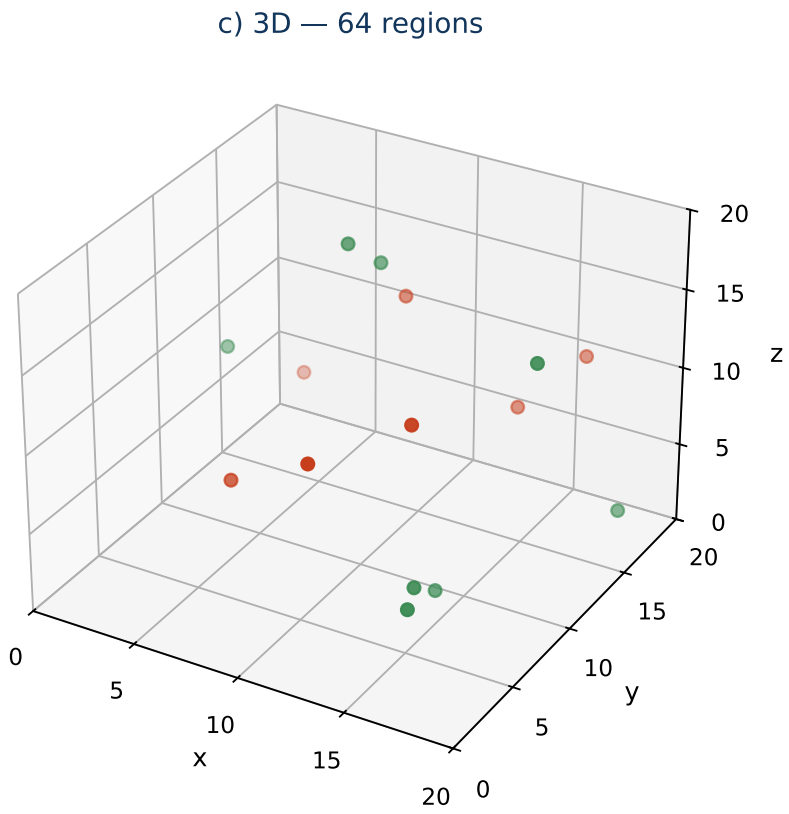
Department of Biomedical Engineering · FEEC, Brno University of Technology

2026

Curse of Dimensionality





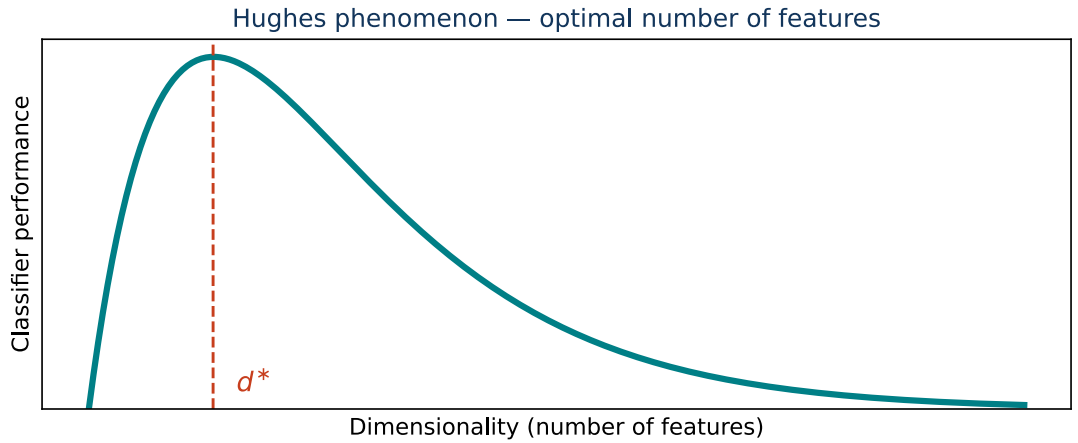


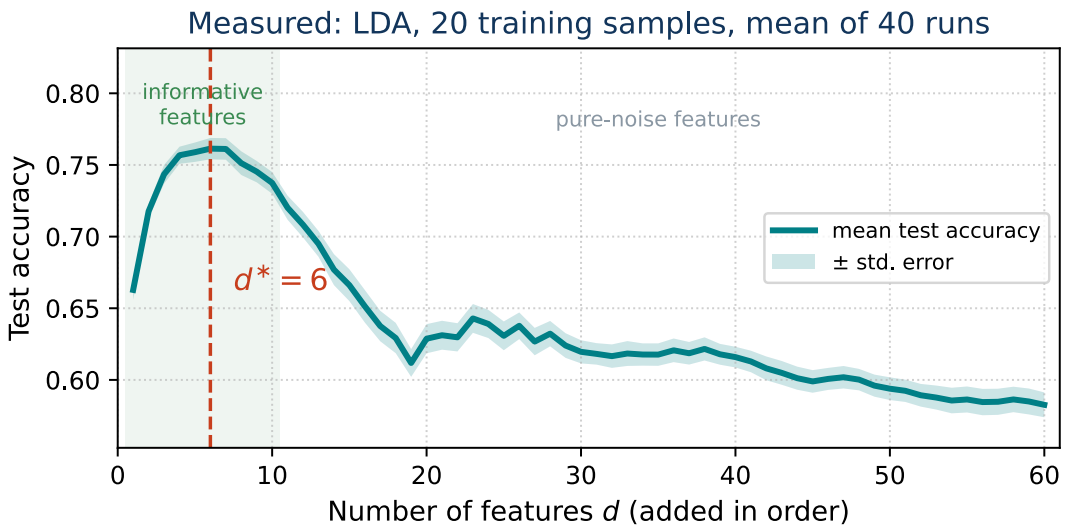
With **4 bins per feature**, d features give 4^d cells. For a fixed sample size n, the average occupancy is $n/4^d$.

$\rightarrow$ need more training data to fill this space

Hughes Phenomenon

With *limited* training data, adding features can eventually **hurt prediction**: the information gained may not compensate for harder estimation.





Key idea: The best subset size depends on the **data and model**; an interior optimum is possible, not inevitable. Right: measured on synthetic data — 2 classes, 20 training samples, features added one by one (first 10 informative, the rest noise).

$\rightarrow$ reducing number of features can improve model! But which features to select?

Feature Selection vs. Feature Reduction

Feature selection — choose a subset of original features

- **Filter methods** — model independent, statistical
  - Mutual information, correlation, mRMR
- **Wrapper methods** — use model performance
  - Forward selection, backward elimination
- **Embedded methods** — built into model training
  - LASSO / Elastic Net, decision trees, random forest

Feature reduction (extraction) — create new features from originals

- **Linear:** PCA, LDA
- **Non-linear:** t-SNE, UMAP, kernel PCA, autoencoders

> **Key difference:** Selection keeps part of the original feature set; reduction builds new features.

Filter methods

> Score features by statistical association, without fitting a predictor.

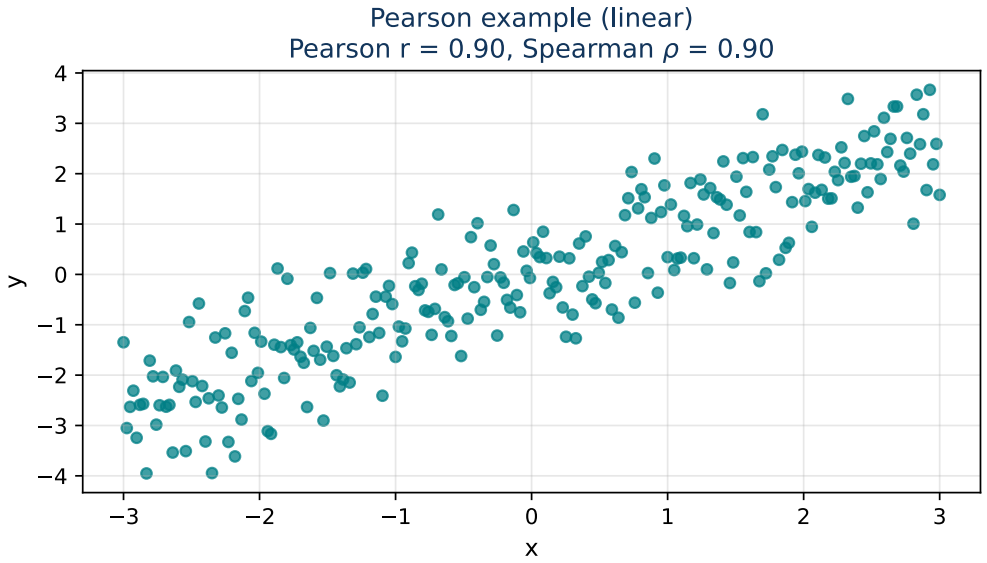
Correlation Coefficients

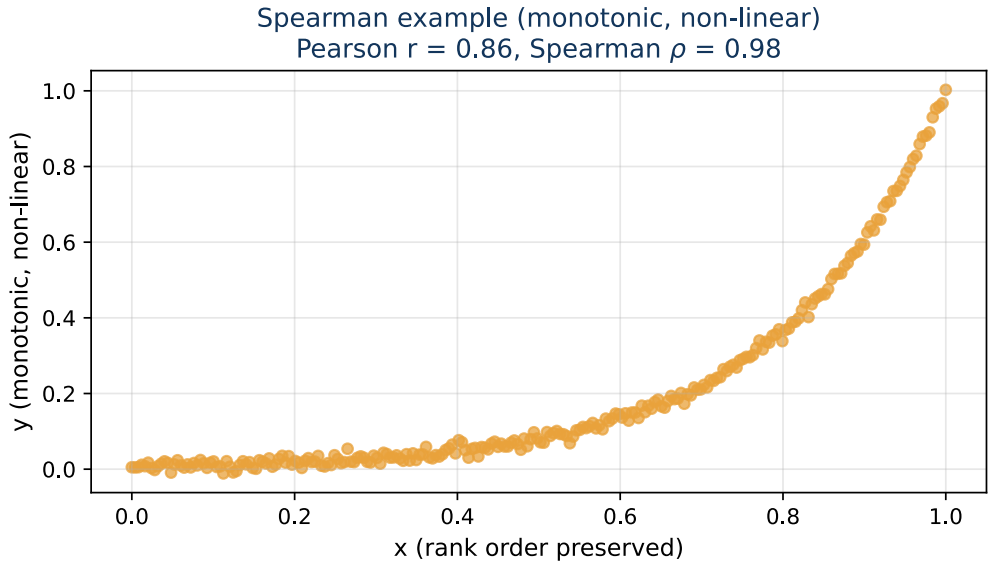
Pearson correlation

- Measures **linear** dependence.
- Sensitive to outliers.
- Works with numeric variables, including discrete ones.
- $\rho_P \in [-1, 1]$; 0 = no linear relation.

Spearman correlation

- Measures **monotonic** dependence.
- Uses ranks; less sensitive to extreme magnitudes.
- Works also for ordinal variables.
- $\rho_S \in [-1, 1]$; 0 does not imply independence.





Use Pearson for linear association; Spearman for monotonic association or ordinal data.

Pearson vs. Spearman Correlation

Pearson = joint variation / scale normalization

$$\rho_P(x,y) = \frac{\operatorname{cov}(x,y)}{\sigma_x \sigma_y} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}.$$

Spearman = the same calculation on ranks

$$\rho_S(x,y) = \rho_P(\operatorname{rank}(x), \operatorname{rank}(y)).$$

Value	Interpretation	Typical example
Near **+1**	Increase together	Height and weight
Near **−1**	One rises, the other falls	Temperature and heating costs
Near **0**	Little linear / monotonic association	Shoe size and exam score

$\rho \in [-1, 1]$. The sample factors $1/(n-1)$ cancel in the ratio.

Pearson vs. Spearman by Hand

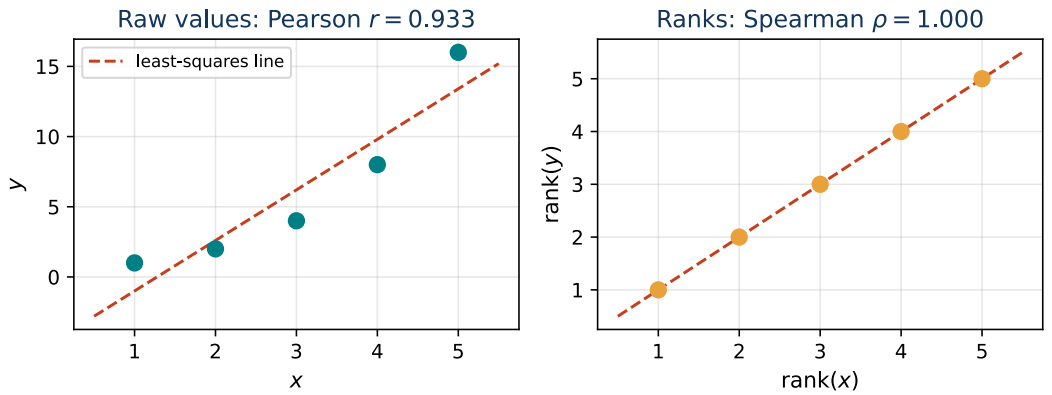
Five samples, y doubles every step ($y = 2^{x-1}$):

x	y	$x - \bar{x}$	$y - \bar{y}$	rank x	rank y
1	1	-2	-5.2	1	1
2	2	-1	-4.2	2	2
3	4	0	-2.2	3	3
4	8	1	1.8	4	4
5	16	2	9.8	5	5

$\bar{x} = 3, \bar{y} = 6.2$; the $\frac{1}{n-1}$ factors cancel:

$$\rho_P = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

$$= \frac{36}{\sqrt{10 \cdot 148.8}} = \frac{36}{38.57} = 0.933$$



Ranks are identical $\Rightarrow \rho_S = \rho_P(\text{rank}\, x, \text{rank}\, y) = 1.$

Perfectly monotonic, but not linear: Spearman says 1, Pearson only 0.93. Try it: swap the last two y values — the ranks change, ρ_S drops to 0.9, ρ_P to 0.73.

Entropy

One variable: uncertainty of an outcome

$$H(X) = -\sum_{x} p(x) \log_2 p(x).$$

Two variables: uncertainty of a pair

$$H(X, Y) = -\sum_{x,y} p(x, y) \log_2 p(x, y).$$

After observing Y: remaining uncertainty of X

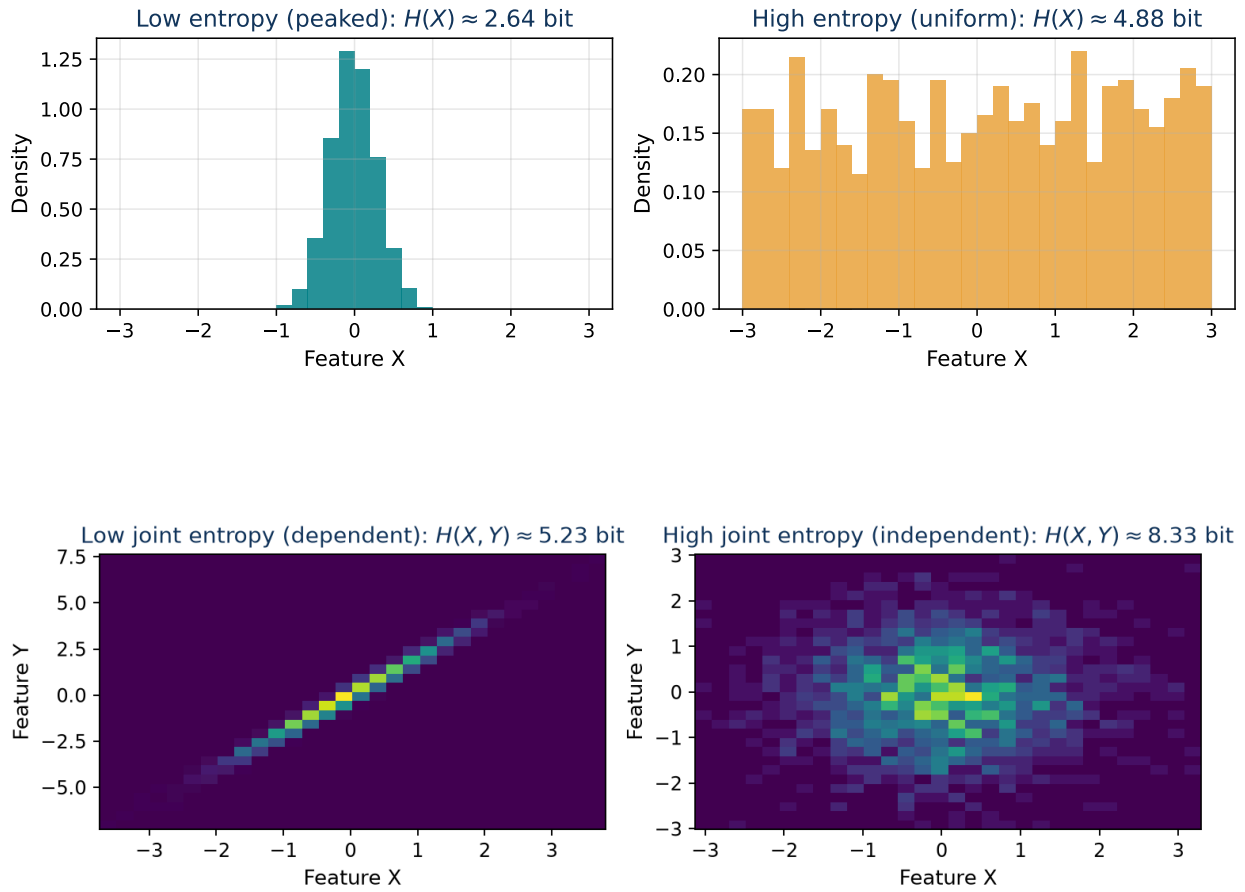
$$H(X|Y) = -\sum_{y} p(y) \sum_{x} p(x|y) \log_2 p(x|y).$$

Examples in bits

- Fair coin: **1**.
- Biased coin ($p = 0.9$): **0.469**.
- Fair die: $\log_2 6 = \mathbf{2.585}$.
- Certain outcome: **0**.

Estimate discrete probabilities by counting outcomes or pairs; histograms discretize continuous data. code example

Entropy histograms

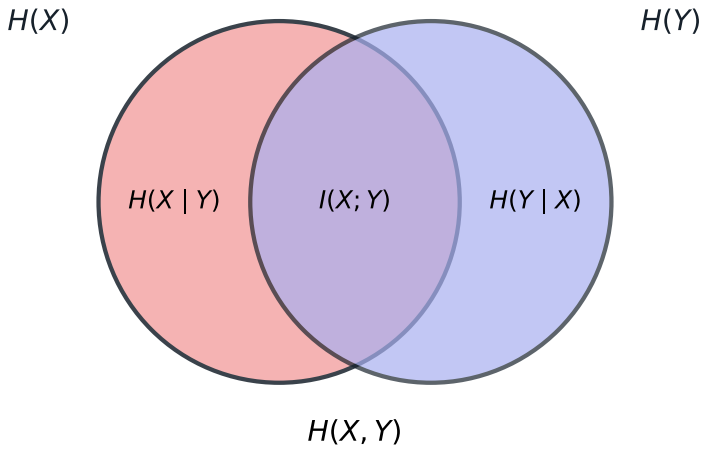


Few occupied outcomes → less uncertainty; many evenly occupied outcomes → more. Same 30 bins per axis; 1D range $[-3, 3]$. Maxima: 4.91 bits (1D), 9.81 bits (2D).

Mutual Information

Information gained = uncertainty before − uncertainty after

$$I(X;Y) = H(Y) - H(Y|X).$$



Equivalent forms

$$\begin{aligned} I(X;Y) &= H(X) - H(X|Y) \\ &= H(X) + H(Y) - H(X,Y). \end{aligned}$$

Discrete variables: from probabilities

$$I(X;Y) = \sum_{x,y} p(x,y) \log_2 \frac{p(x,y)}{p(x)p(y)}.$$

Captures nonlinear dependence and works with categorical variables too.

Mutual Information in Practice

Both variables discrete: count pairs

1. Build a count table: n_{xy} observations of each pair.
2. Divide by n; sum rows / columns for marginals:

$$\hat{p}(x, y) = \frac{n_{xy}}{n}, \qquad \hat{p}(x) = \sum_{y} \hat{p}(x, y).$$

3. Substitute into the MI sum; skip zero-count cells.

Continuous data: estimate local density

- **kNN:** neighbour distances and counts measure local concentration; choose k.
- **KDE:** smooth bumps around samples estimate densities; choose bandwidth.
- **Binning:** discretize, then count pairs; bin edges and widths matter.

Mixed pair: use an estimator for continuous values within discrete groups.

scikit-learn: choose by target type. Class labels → `mutual_info_classif`; continuous target → `mutual_info_regression`. Mark input types with `discrete_features`.

scikit-learn returns **nats**; divide by $\ln 2$ for bits. code example

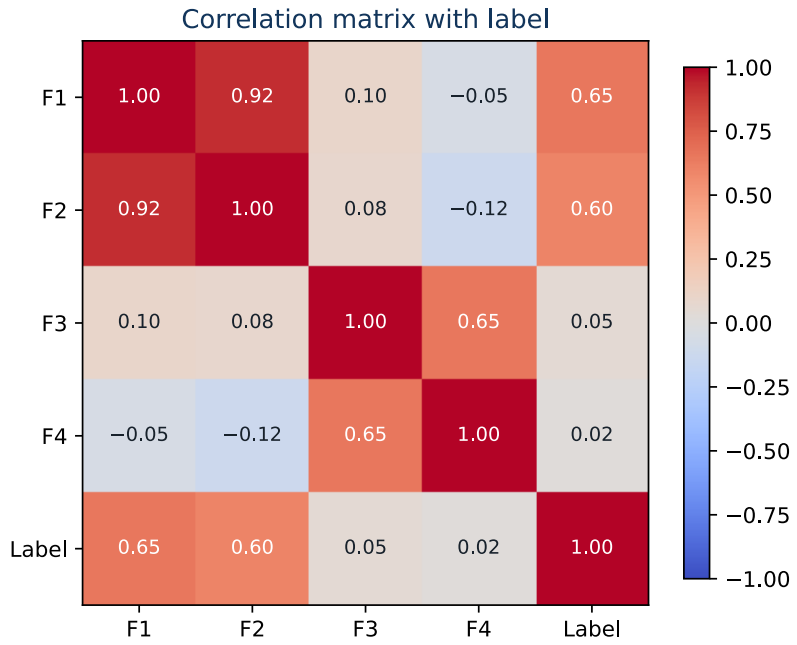
Correlation Matrix with Label

Idea: By adding the **label** to the correlation matrix we can see

- which features are strongly related to the target (**relevance**),
- which features are strongly related to each other (**redundancy**).

Rows / columns: F_1, F_2, F_3, F_4, Y.

$$\begin{bmatrix} 1.00 & 0.92 & 0.10 & -0.05 & 0.65 \\ 0.92 & 1.00 & 0.08 & -0.12 & 0.60 \\ 0.10 & 0.08 & 1.00 & 0.65 & 0.05 \\ -0.05 & -0.12 & 0.65 & 1.00 & 0.02 \\ 0.65 & 0.60 & 0.05 & 0.02 & 1.00 \end{bmatrix}$$



Selection rule: useful information, without duplication.

- **Relevant:** F_1–Y: **0.65**; F_2–Y: **0.60**.
- **Redundant:** F_1–F_2: **0.92** — they largely overlap.

mRMR: Minimum Redundancy, Maximum Relevance

- **Relevance:** information about the target.
- **Redundancy:** information already present in selected features.

Start with the feature having the largest $I(X_j; Y)$. For a nonempty selected set $\mathcal{S}$: **candidate relevance − average overlap**.

$$j^\star = \arg\max_{j \notin \mathcal{S}} \left[\underbrace{I(X_j; Y)}_{\text{relevance}} - \underbrace{\frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} I(X_j; X_i)}_{\text{average redundancy}} \right].$$

Add $j^\star$ to $\mathcal{S}$ until the chosen budget k is reached.

A **greedy filter**: use MI here, not the preceding correlation values; no predictor is fitted.

code example

Wrapper methods

> Let the model itself say which subset of features works — at the price of training it many times over.

Wrapper Selection Is Optimization

Choose a subset S of the feature indices $\mathcal{F} = \{1, \ldots, d\}$:

$$S^\star = \arg \min_{\substack{S \subseteq \mathcal{F} \\ |S| = k}} \underbrace{\hat{R}_{\mathrm{CV}}(\mathcal{P}_S)}_{\text{validation loss after fitting on features } S}$$

- **Decision:** which k features to keep — a discrete search space.
- **One evaluation:** fit preprocessing + model inside each CV training fold.
- **As in the second lecture:** choose a pipeline; fit model parameters separately for each candidate.

> Accuracy as the score? **Maximize** CV accuracy, equivalently minimize $1 - \mathrm{accuracy}$.

One Objective, Different Search Strategies

Strategy	Which subsets do we evaluate?	Trade-off
Brute force	Every feasible subset	Best CV score; combinatorial cost
Random search	A budget of randomly sampled subsets	Broad exploration; no optimum guarantee
Greedy heuristic	All one-feature moves from the current subset	Cheap local steps; decisions are not revisited

- **Forward:** start empty, repeatedly add the best candidate.
- **Backward:** start full, repeatedly remove the best candidate.

Same objective, different search paths — neither greedy direction guarantees the best subset. Definitions and worked example.

Forward Selection: Steps, Trials and Evaluation

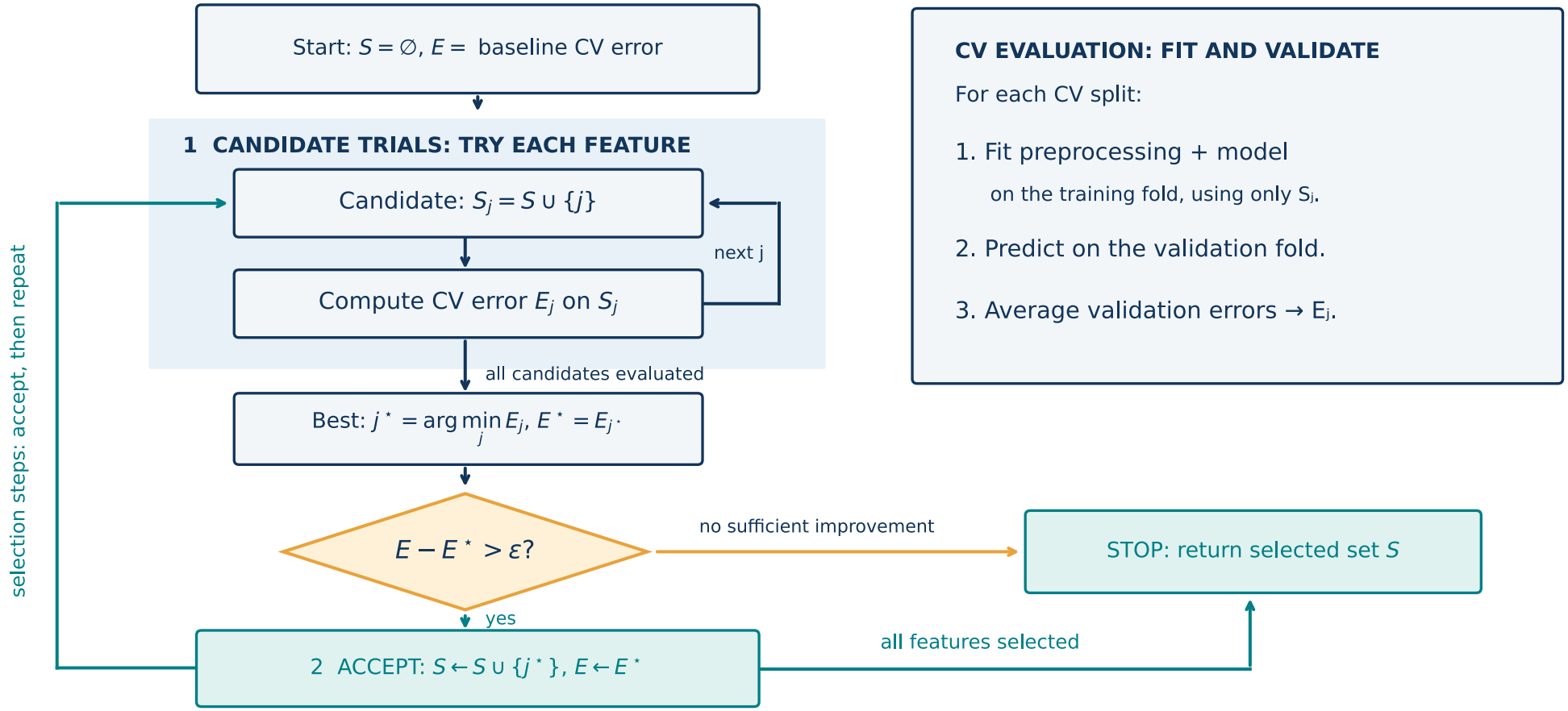
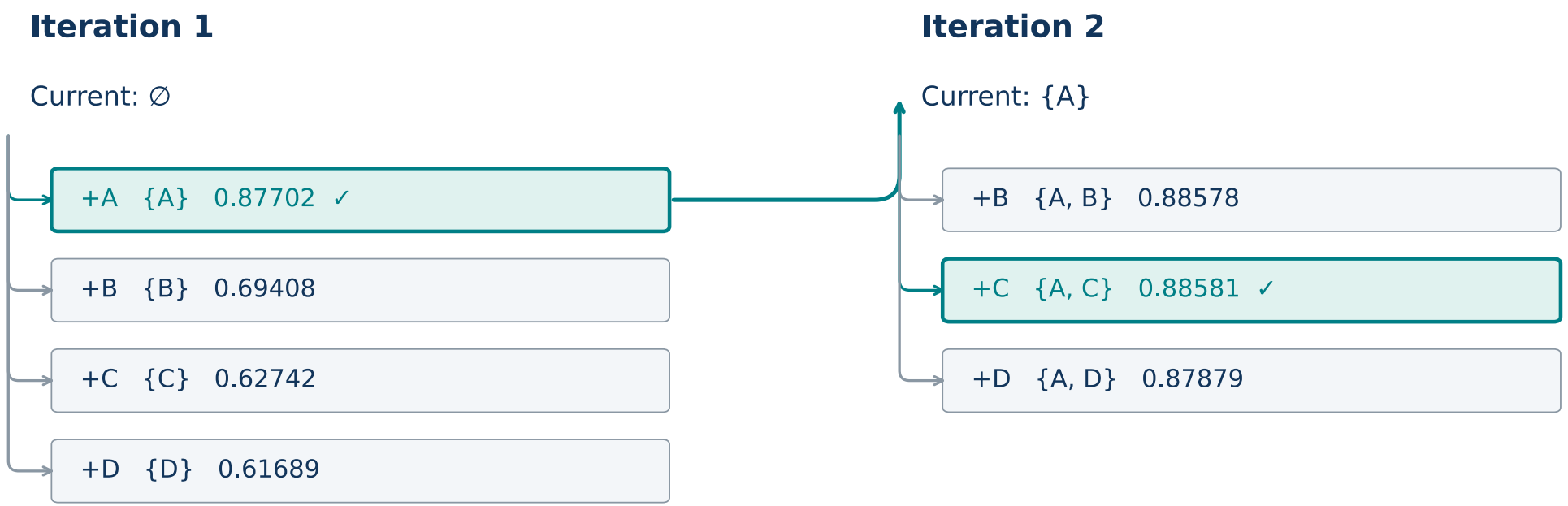


Diagram: stop on insufficient improvement; baseline = dummy predictor. **Next example:** stop at $k = 2$.

Forward Selection: Try Every Addition



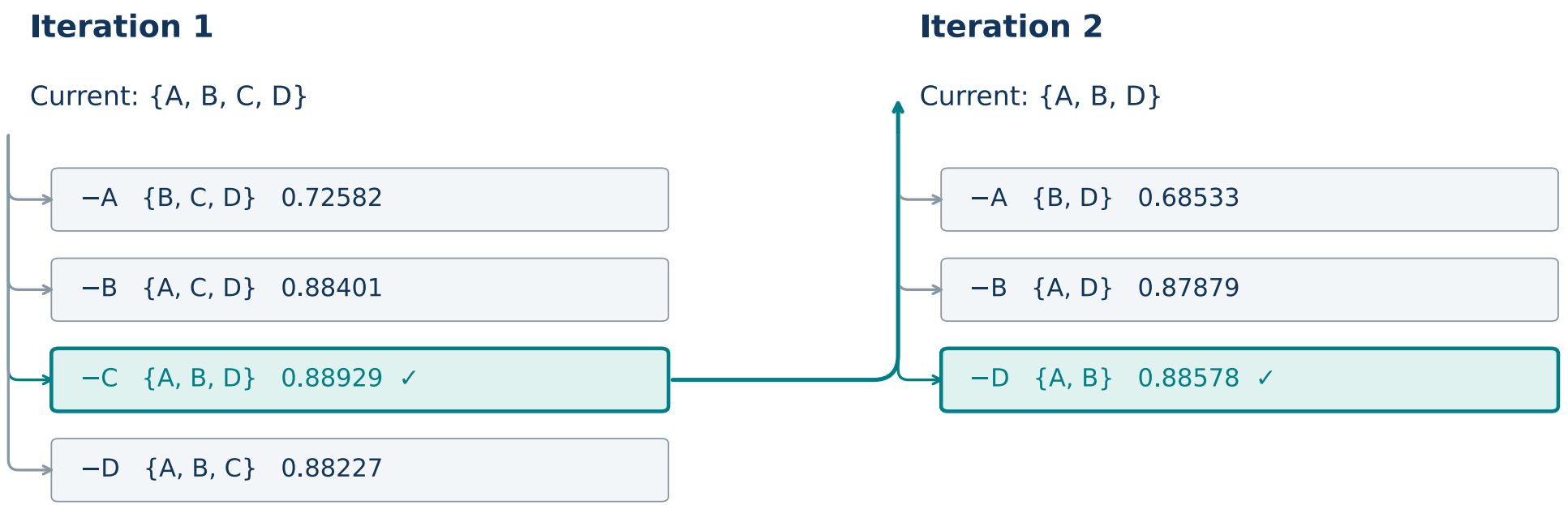
A: mean radius · B: mean texture · C: texture error · D: fractal dimension error

Numbers: mean CV accuracy | Green: keep this candidate | Stop at 2 features

Greedy step: $j^\star = \arg\max_{j \notin S} \mathrm{CVAcc}(S \cup \{j\})$, then $S \leftarrow S \cup \{j^\star\}$. **Stop here:** $|S| = 2$. Added features stay selected.

Breast cancer data · logistic regression + scaling inside 5-fold CV · search scores, not test results. code example

Backward Elimination: Try Every Removal



A: mean radius · B: mean texture · C: texture error · D: fractal dimension error

Numbers: mean CV accuracy | Green: keep this candidate | Stop at 2 features

Greedy step: $j^\star = \arg\max_{j \in S} \mathrm{CVAcc}(S \setminus \{j\})$, then $S \leftarrow S \setminus \{j^\star\}$. **Stop here:** $|S| = 2$. Removed features cannot return.

Same data, model and CV splits · evaluate the remaining subset, not the removed feature alone. code example

Embedded Methods

> Some models pick their features while they are being fitted — and then we move from selecting features to building new ones.

Embedded Methods

Idea: Feature selection happens **during model training** — the model itself decides which features are useful.

Examples:

- **Regularization:** LASSO (L1), Elastic Net
  - Can set coefficients exactly to zero.
- **Tree-based models:** decision trees, random forest, gradient boosted trees
  - Select features at splits; noise can also be selected.
- **Linear models with penalties:** SVM with L1 penalty, logistic regression with L1 or Elastic Net.

Pros: Selection is part of fitting; avoids a separate subset-search loop.

Cons: Model-specific; selected features may depend on model choice/hyperparameters.

Will be explained with individual models.

Feature reduction

> Build a smaller set of new features from the original measurements.

Feature Reduction – General View

Goal: Replace d original features by $k \ll d$ new features, while preserving as much information as possible.

In simple terms: New features are created as a “mixture” of old features.

General mapping:

$$\mathbf{x} \in \mathbb{R}^d \quad \mapsto \quad \mathbf{z} \in \mathbb{R}^k$$

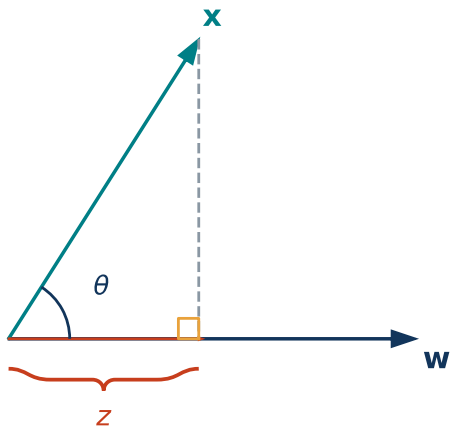
- **Linear reduction:** $\mathbf{z} = W^\top \mathbf{x},\ W \in \mathbb{R}^{d \times k}$ — new features are weighted sums of the originals. *Examples: PCA, LDA, ICA*
- **Non-linear reduction:** $\mathbf{z} = f(\mathbf{x}), f$ is non-linear (NN, kernels, etc.) — can capture curved manifolds, local neighborhoods. *Examples: t-SNE, UMAP, autoencoders*

> **Key trade-off:** linear = simple, interpretable; non-linear = more flexible, but less transparent.

Linear reduction

> Use weighted sums of features to describe the data in fewer dimensions.

Inner Product (Dot Product) and Projection



Dot product (inner product):

$$z = \mathbf{x} \cdot \mathbf{w} = \mathbf{x}^\top \mathbf{w} = \langle \mathbf{x}, \mathbf{w} \rangle = \|\mathbf{x}\| \, \|\mathbf{w}\| \, \cos\theta$$

If $\|\mathbf{w}\| = 1$ (unit vector), then z is the signed length of the projection of $\mathbf{x}$ onto $\mathbf{w}$.

Matrix view (multiple directions):

$$\mathbf{z} = W^\top \mathbf{x} \in \mathbb{R}^k, \qquad z_j = \mathbf{w}_j^\top \mathbf{x}$$

Each component z_j is a dot product (projection) onto column $\mathbf{w}_j$ of W. It is also a linear transformation of the vector $\mathbf{x}$.

Linearity Recap

Linearity rules:

- **Additivity:** $f(\mathbf{x}_1 + \mathbf{x}_2) = f(\mathbf{x}_1) + f(\mathbf{x}_2)$
- **Homogeneity:** $f(a\mathbf{x}) = af(\mathbf{x})$
- $\rightarrow$ **Principle of superposition:** $f(a\mathbf{x}_1 + b\mathbf{x}_2) = af(\mathbf{x}_1) + bf(\mathbf{x}_2),\ \forall a, b \in \mathbb{R}.$

A linear function has to satisfy both rules — and the only functions that do are weighted sums of the inputs.

Implication for linear reduction: new features are **linear mixtures** (superpositions) of the originals.

Example: one new feature z_1

$$z_1 = w_{1,1}x_1 + w_{2,1}x_2 + \cdots + w_{d,1}x_d, \qquad \text{e.g. } z_1 = 0.7x_1 - 0.2x_2 + 0.5x_3$$

x_j = original features; $w_{j,1}$ = weight of feature x_j in the new feature z_1.

Matrix form (all new features): $\mathbf{z} = W^\top \mathbf{x},\ W = [\mathbf{w}_1, \ldots, \mathbf{w}_k]$

Why Linearity is Useful

$$\mathbf{z} = W^\top \mathbf{x}$$

Simple mathematics

One matrix multiplication; standard linear algebra.

Readable weights

Each component is a weighted sum of the inputs.

Clear geometry

Rotations, scalings and projections; lines stay lines or collapse to points.

Efficient computation

Optimized matrix operations; PCA via eigenvectors or SVD.

A useful starting point: try a linear representation before adding complexity.

Principal component analysis

> Find directions that preserve the most variance.

Principal Component Analysis (PCA) – Introduction

Unsupervised: PCA learns from X alone — no target labels required.

Goal: Project data onto fewer **uncorrelated components** that retain as much variance as possible.

Intuition:

- High-dimensional data often has redundant / correlated features.
- PCA finds new **axes (directions)** where the data varies the most.
- First PC = direction of maximum variance. Second PC = orthogonal to the first, with the next-largest variance, etc.

Why use PCA?

- Compress data with minimal squared reconstruction error.
- Denoise when discarded directions mainly contain noise.
- **Explore and visualize data:** 2D/3D views, trends and unusual samples.
- **Analyze feature relationships:** inspect component weights and biplots.
- **Preprocessing:** optional input to clustering or supervised models.

Projection and Projection Error

Setup: Data points $\mathbf{x}_i \in \mathbb{R}^d, i = 1, \ldots, n$, are centered at the origin. We look for a unit direction $\mathbf{w}$ (a line through the origin).

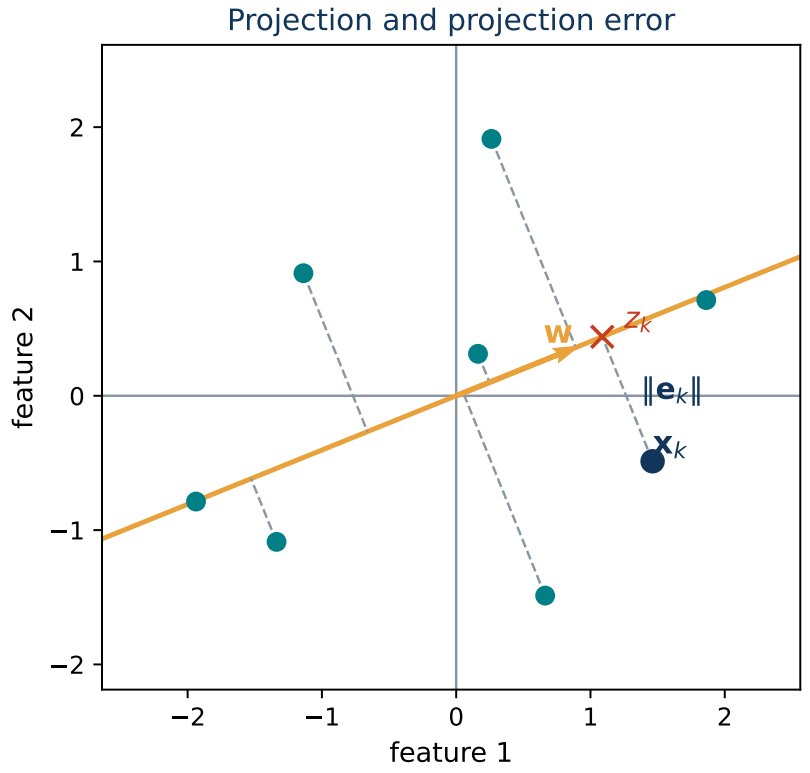
Projection of $\mathbf{x}_i$ on $\mathbf{w}$:

$$z_i = \mathbf{w}^\top \mathbf{x}_i \quad \Longrightarrow \quad \hat{\mathbf{x}}_i = z_i \, \mathbf{w}$$

z_i = coordinate of $\mathbf{x}_i$ along $\mathbf{w}$; $\hat{\mathbf{x}}_i$ = reconstructed point on the line.

Projection (reconstruction) error:

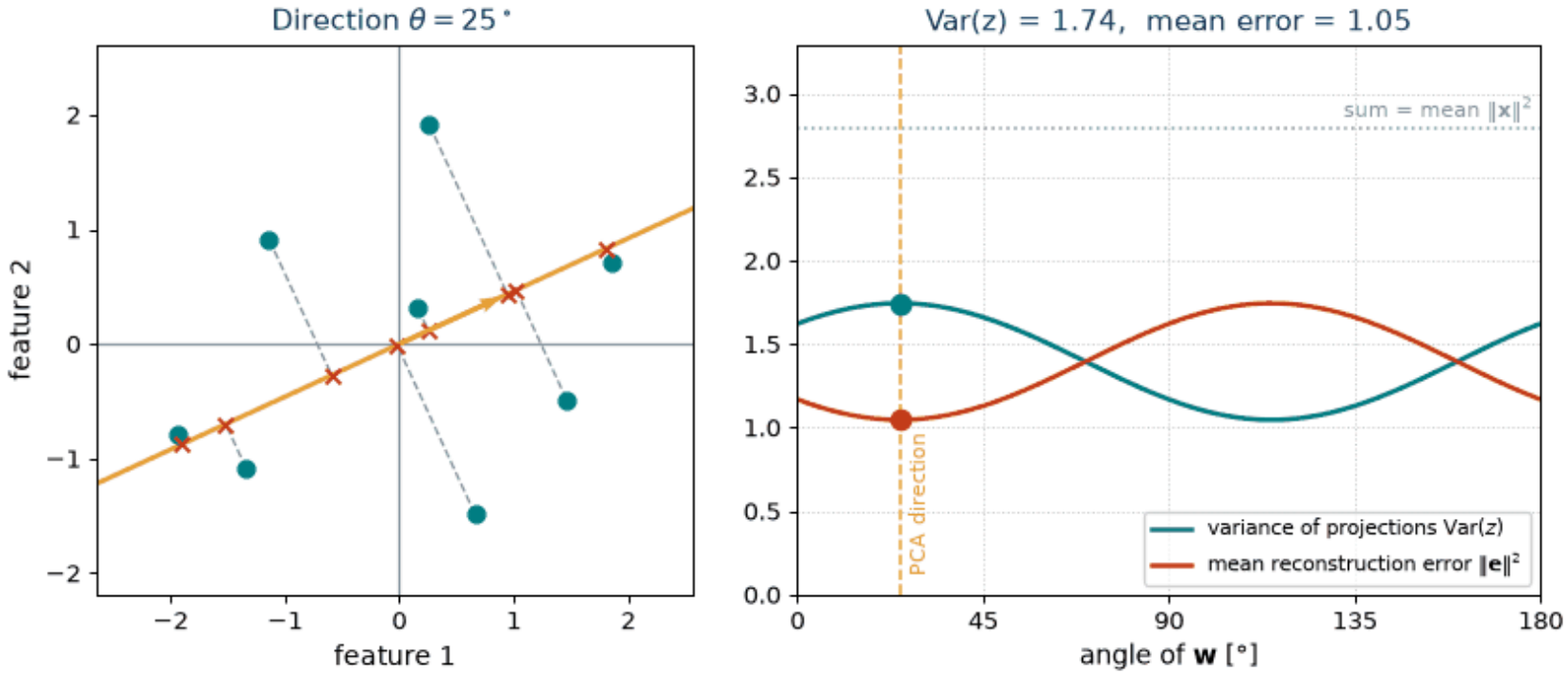
$$\mathbf{e}_i = \mathbf{x}_i - \hat{\mathbf{x}}_i, \qquad \|\mathbf{e}_i\|^2 = \|\mathbf{x}_i\|^2 - z_i^2$$



Why maximize the variance of projections?

- Maximizing $\mathrm{Var}(z_i)$ $\Leftrightarrow$ minimizing the average reconstruction error $\|\mathbf{e}_i\|^2$.
- Intuition: choose the line where projected points are most "spread out" $\Leftrightarrow$ the line that yields the smallest projection error.

Rotating the Direction: Variance vs. Error



Same eight centered points, every direction $\mathbf{w}$ tried in turn. For every angle $\mathrm{Var}(z) + \frac{1}{n}\sum_i \|\mathbf{e}_i\|^2 = \frac{1}{n}\sum_i \|\mathbf{x}_i\|^2 \Rightarrow$ **largest variance** $\Leftrightarrow$ **smallest error**. The maximum $\mathrm{Var}(z) = 1.74$ is the largest eigenvalue of S (next slides).

From Projection Error to Variance Maximization

For centred inputs, a unit direction $\mathbf{w}$ and scores $z_i = \mathbf{w}^\top \mathbf{x}_i$:

$$\underbrace{\|\mathbf{x}_i\|^2}_{\text{original vector}} = \underbrace{z_i^2}_{\text{projection}} + \underbrace{\|\mathbf{e}_i\|^2}_{\text{reconstruction error}}.$$

Average over the samples:

$$\frac{1}{n}\sum_i \|\mathbf{e}_i\|^2 = \underbrace{\operatorname{trace}(S)}_{\text{total variance: fixed}} - \underbrace{\mathbf{w}^\top S \mathbf{w}}_{\text{projected variance}}, \qquad S = \frac{1}{n} X^\top X.$$

Minimum squared reconstruction error = maximum retained variance.

PCA as an Optimization Problem

Setup (centered data): Let $X \in \mathbb{R}^{n \times d}$ be zero-mean (each column centered). Empirical covariance (divisor n): $S = \frac{1}{n} X^\top X \in \mathbb{R}^{d \times d}$.

First principal component (direction $\mathbf{w}_1$):

$$\max_{\mathbf{w}_1 \in \mathbb{R}^d} \operatorname{Var}(X\mathbf{w}_1) = \max_{\mathbf{w}_1} \mathbf{w}_1^\top S \mathbf{w}_1 \quad \text{s.t.} \quad \|\mathbf{w}_1\|_2 = 1.$$

Top-k components (matrix $W \in \mathbb{R}^{d \times k}$):

$$\max_{W \in \mathbb{R}^{d \times k}} \operatorname{trace}\left(W^\top S W\right) \quad \text{s.t.} \quad W^\top W = I_k.$$

Notes:

- Orthonormality prevents repeated directions; **covariance eigenvectors** give uncorrelated scores.
- Centering X is essential; otherwise S mixes mean effects with variance.

PCA Solution via Eigen-Decomposition

Problem (recap): $\max_{\mathbf{w}\in\mathbb{R}^d} \mathbf{w}^\top S\mathbf{w}$ s.t. $\|\mathbf{w}\|_2 = 1$.

How to solve it? We use a common optimization trick: put the constraint directly into the function we optimize (Lagrangian).

Lagrangian:

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^\top S\mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1)$$

At a constrained optimum, the Lagrangian derivative is zero.

Stationarity:

$$\nabla_{\mathbf{w}}\mathcal{L} = 2S\mathbf{w} - 2\lambda\mathbf{w} = 0 \quad \Longrightarrow \quad S\mathbf{w} = \lambda\mathbf{w}$$

Which eigenvector maximizes variance? For a unit eigenvector:

$$\mathbf{w}^\top S\mathbf{w} = \lambda\mathbf{w}^\top\mathbf{w} = \lambda.$$

Choose the **largest eigenvalue**: it is the retained variance.

> For top-k components, the solution is just more eigenvectors with the largest eigenvalues.

PCA by Hand: Four Points

1. Centre and form the covariance (column means are already 0):

$$X = \begin{bmatrix} 2 & 1 \\ -2 & -1 \\ 1 & 2 \\ -1 & -2 \end{bmatrix}, \qquad S = \tfrac{1}{4} X^\top X = \tfrac{1}{4} \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix} = \begin{bmatrix} 2.5 & 2 \\ 2 & 2.5 \end{bmatrix}$$

2. Find directions and variances

$\det(S - \lambda I) = (2.5 - \lambda)^2 - 4 = 0$:

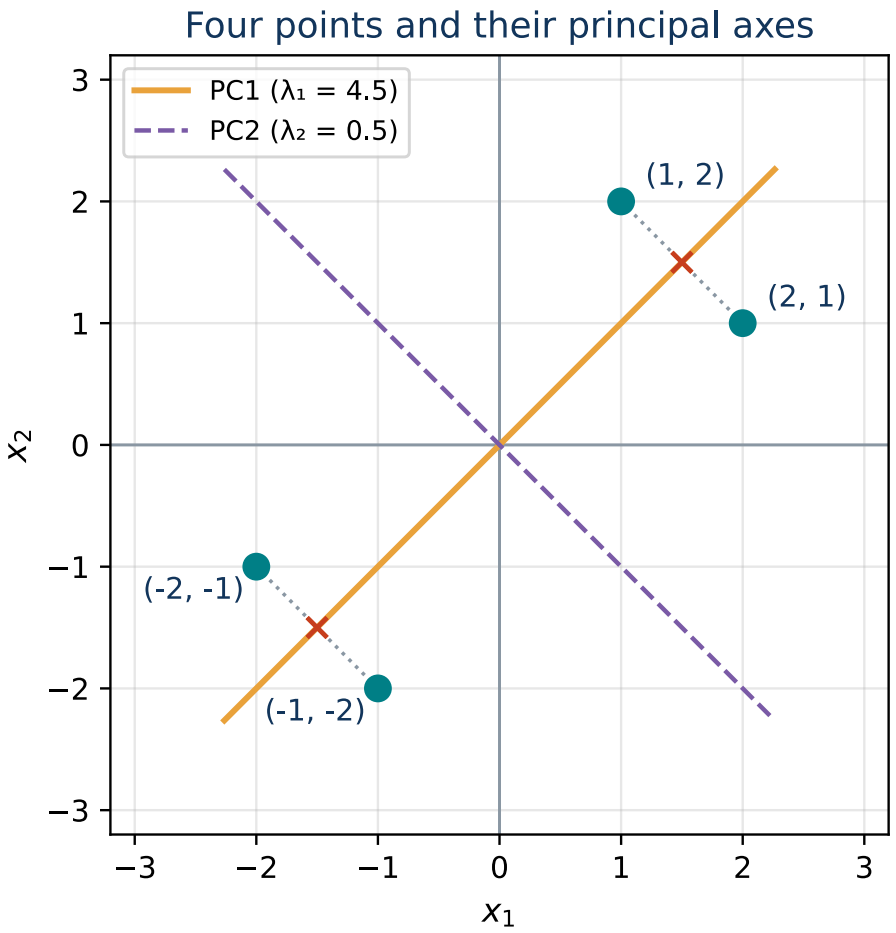
$$\lambda_1 = 4.5, \quad \mathbf{w}_1 = \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}; \qquad \lambda_2 = 0.5, \quad \mathbf{w}_2 = \tfrac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

3. Project: first score $(2 + 1)/\sqrt{2} = 3/\sqrt{2}$.

$$\mathbf{z}_1 = X\mathbf{w}_1 = \tfrac{1}{\sqrt{2}}(3, -3, 3, -3)^\top = (2.12, -2.12, 2.12, -2.12)^\top$$

4. Check retained variance

$\mathrm{Var}(z_1) = \frac{1}{4} \sum z_{1i}^2 = \frac{1}{4} \cdot 18 = 4.5 = \lambda_1$; explained variance $\lambda_1/(\lambda_1 + \lambda_2) = 90\,\%$.



Using PCA: Exploration and Prediction

PCA itself is unsupervised: its directions are fitted without labels.

Explore the data

- Plot samples in **2D or 3D**.
- Look for trends, groups and unusual samples.
- Inspect **weights** / **biplots** to understand the axes.
- No prediction model is required.

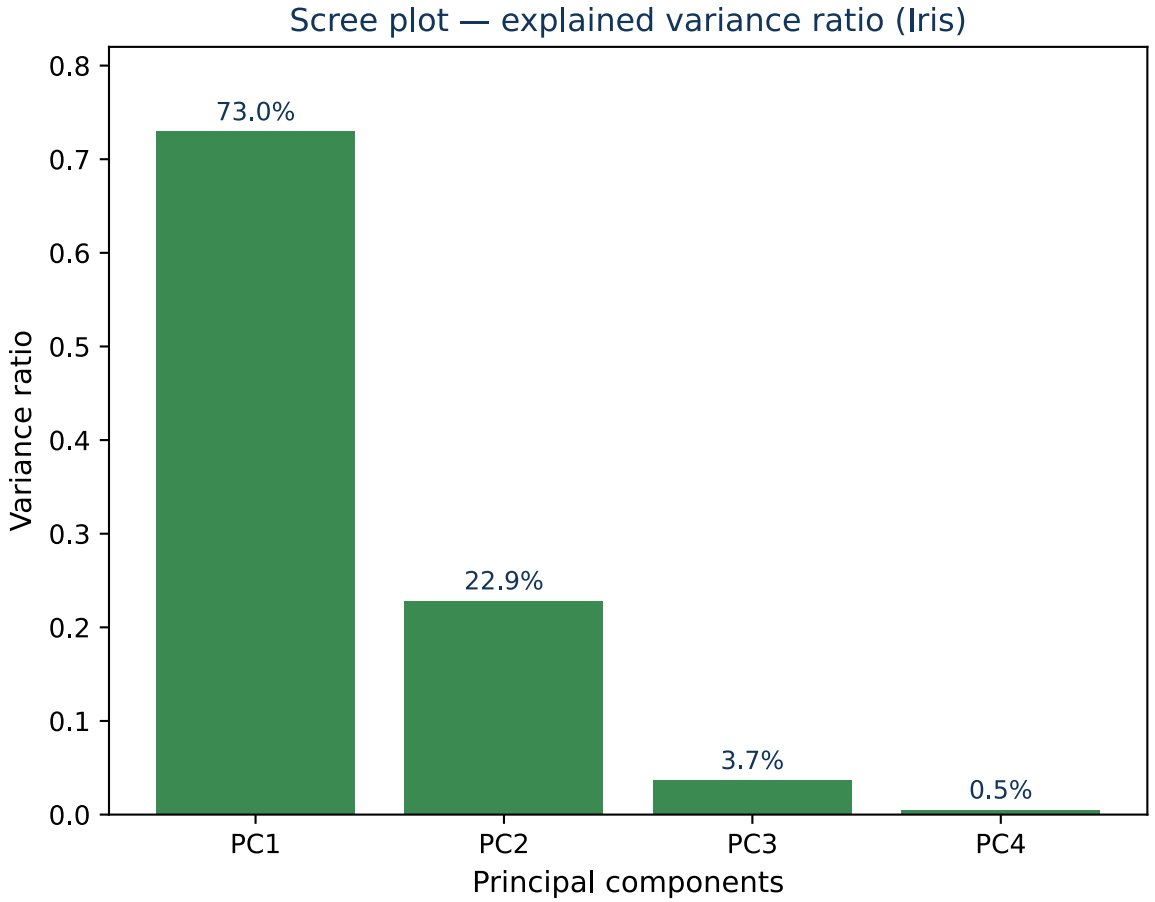
Prepare inputs for prediction

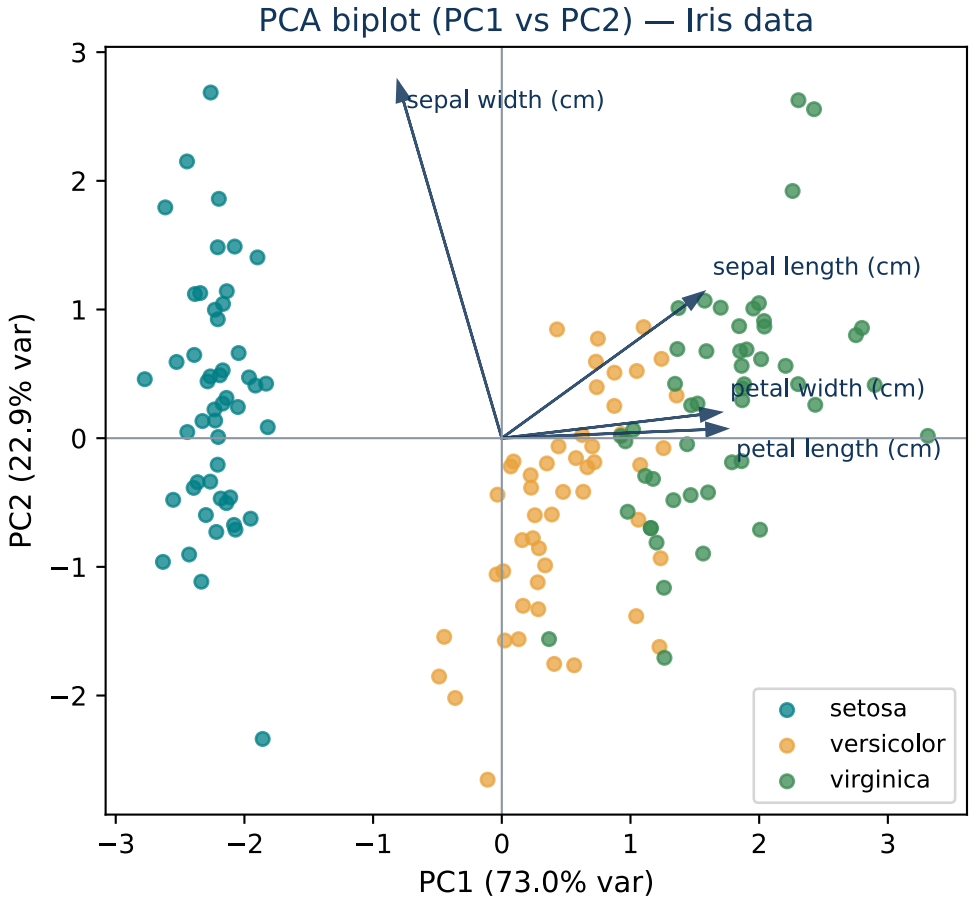
1. Split data / set up CV.
2. Fit scaling + PCA on **training data only**.
3. Transform held-out data with that same fit.
4. Fit a predictor; tune k inside CV.

Different goals: explained variance describes the data; validation scores assess prediction.

Scree plot and Biplot

Unsupervised exploration of Iris: standardized features; labels only colour the points.

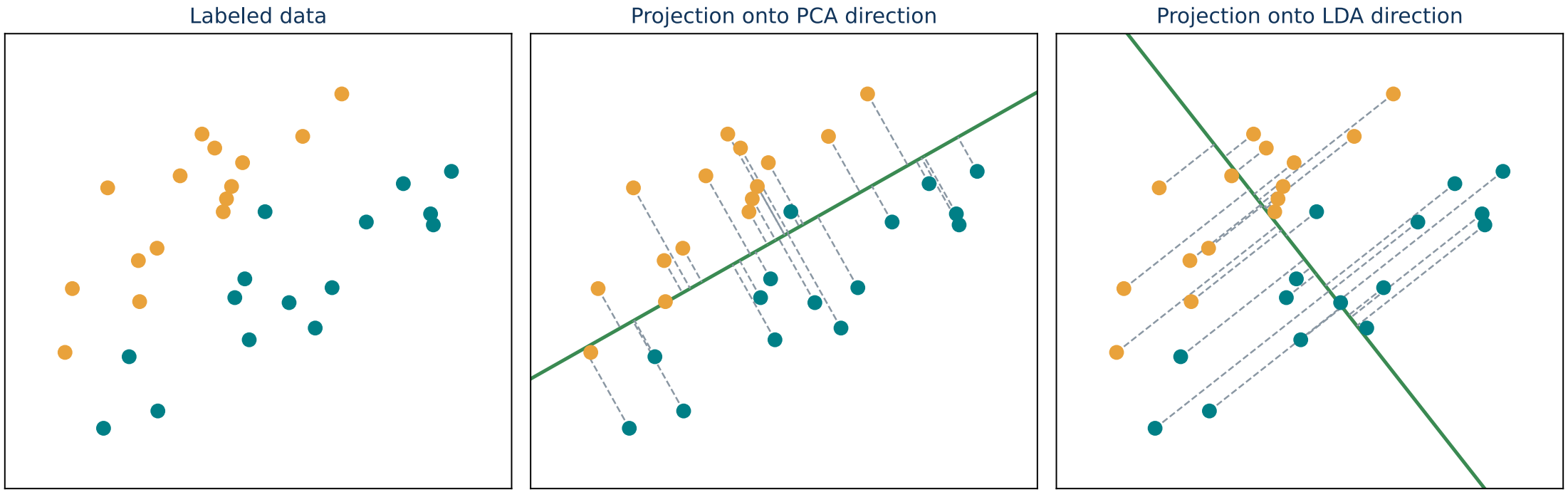




Linear discriminant analysis

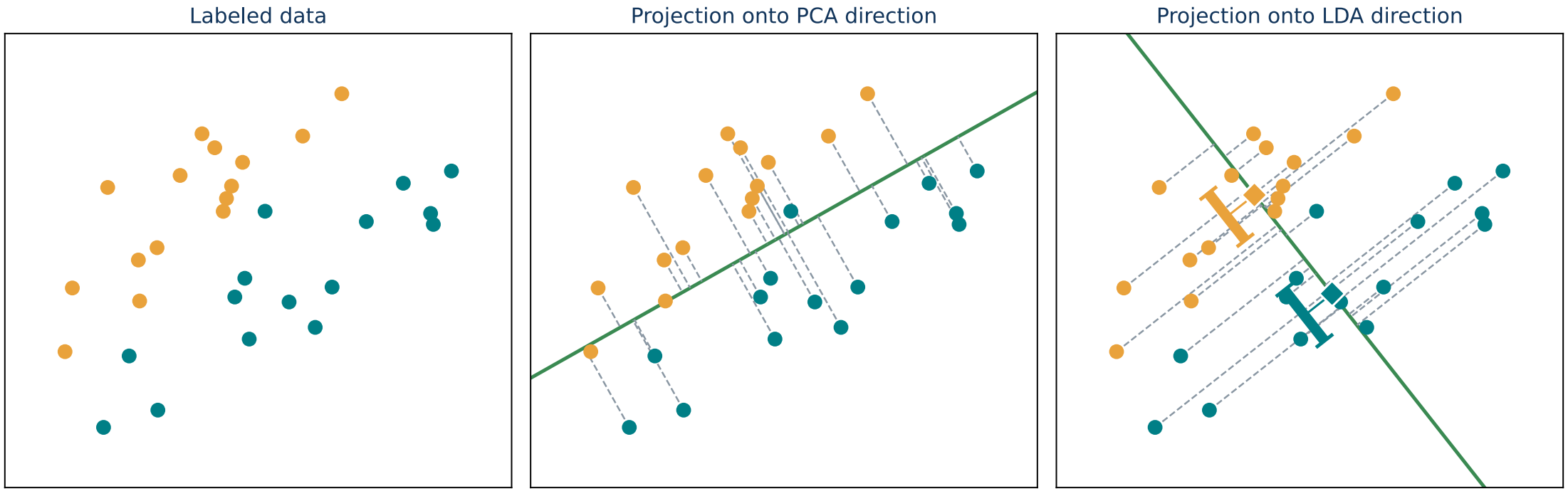
> Use labels to find directions that separate classes.

PCA vs. LDA



- Both are **linear** projections: $Z = XW$ (to a line here; in general to a k-D subspace).
- **PCA** is *unsupervised*: ignores labels and **maximizes variance** (equivalently, *minimizes reconstruction error*).
- **LDA** is *supervised*: uses labels and **maximizes class separation** (between-class ↑, within-class ↓).

LDA: Class Means and Within-Class Spread



Diamond: class mean
Coloured bar: mean ± 1 standard deviation

- **Diamonds:** the mean projected position of each class.
- **Bars:** mean ± one standard deviation; **variance** is the average squared distance from that class mean.
- **LDA:** separate the class means while keeping within-class spread small.

Linear Discriminant Analysis (LDA) — Idea

Compute a separating direction

1. **Class means $\boldsymbol{\mu}_c$.**
2. **Within-class scatter S_W.**
3. Two classes: **maximize Fisher's criterion**

$$\max_{\mathbf{w}} \frac{\left(\mathbf{w}^\top(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)\right)^2}{\mathbf{w}^\top S_W\, \mathbf{w}} \quad \begin{matrix} \leftarrow \text{separation}\uparrow \\ \\ \leftarrow \text{spread}\downarrow \end{matrix}$$

Optimum: solve $S_W \mathbf{w} = \boldsymbol{\mu}_1 - \boldsymbol{\mu}_2$.
Score: $z = \mathbf{w}^\top \mathbf{x}$.

Used in practice

- **Classifier:** practical use, e.g. regularized LDA for EEG signals.
- **Supervised projection:** separates known classes; needs labels.
- **For C classes:** at most $\min(d, C-1)$ directions; not a general replacement for PCA.

For two classes, **one score + a threshold** gives a linear classifier.

Singular S_W: use regularization or a rank-aware solver. Derivation and example · EEG application

Nonlinear reduction

> Preserve local neighbourhoods when a linear projection is not enough.

t-SNE: Keep Neighbours Close

1. **In the original space:** assign stronger similarity to nearby samples.
2. **On a 2D map:** move points so these neighbourhoods are reproduced.
3. **Balance attraction and repulsion:** neighbours gather without everything collapsing.

- `perplexity`**:** effective neighbourhood size; smaller values focus more locally.
- `learning_rate`**:** size of optimization steps; `init` and `random_state` affect the layout.

> Read **local neighbourhoods**. Gaps, island sizes and inter-island distances are not calibrated measurements.

UMAP: Build a Neighbour Graph, Then Lay It Out

Learn the representation

1. Find neighbours in the input space.
2. Build a **weighted neighbour graph**.
3. Embed it in fewer dimensions, preserving strong connections.

Choose the settings

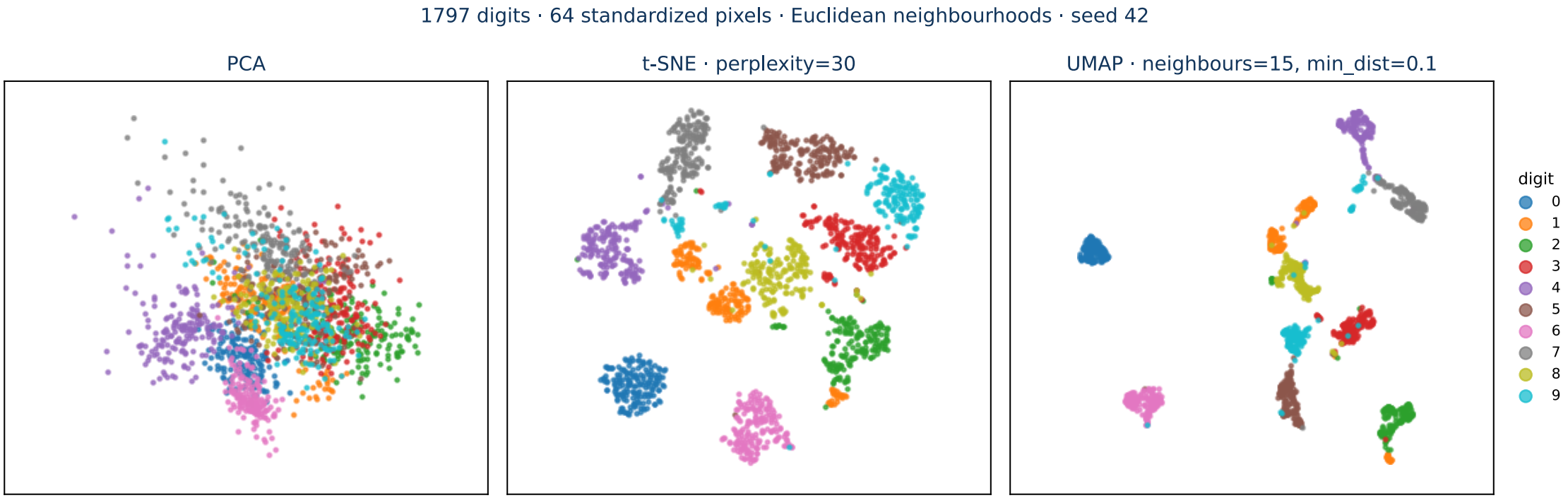
- **`n_neighbors`:** neighbourhood scale.
- **`min_dist`:** output packing.
- **`metric`:** input distance.
- **`n_components`:** output dimension; not limited to 2D.

UMAP has `transform`; scikit-learn t-SNE does not.

UMAP before a classifier: `fit_transform(X_train)` → fit classifier; `transform(X_new)` → predict with the same classifier.

Extensions such as openTSNE also offer new-sample placement. UMAP: transforming new data

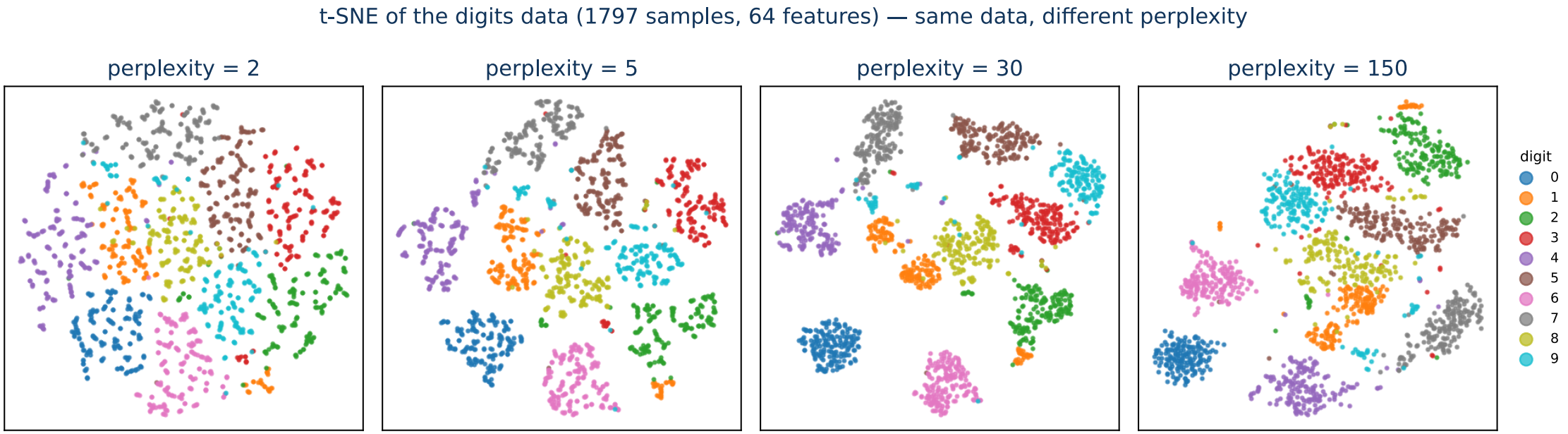
PCA, t-SNE and UMAP: the Same Digits



Same samples and preprocessing; labels only colour the plot. PCA preserves variance; t-SNE and UMAP emphasize neighbourhood structure.

Separate axis scales. Clear islands alone do not establish classes, cluster density or predictive accuracy. code example

t-SNE: the Same Data, Four Perplexities



`perplexity` ≈ effective number of neighbours each point "cares about" (often explored around 5–50). Smaller values emphasize local detail; larger values connect broader neighbourhoods. The useful scale depends on the data. **The layout is not a property of the data alone:** always state the hyperparameters with a t-SNE figure.

Other Reduction Methods

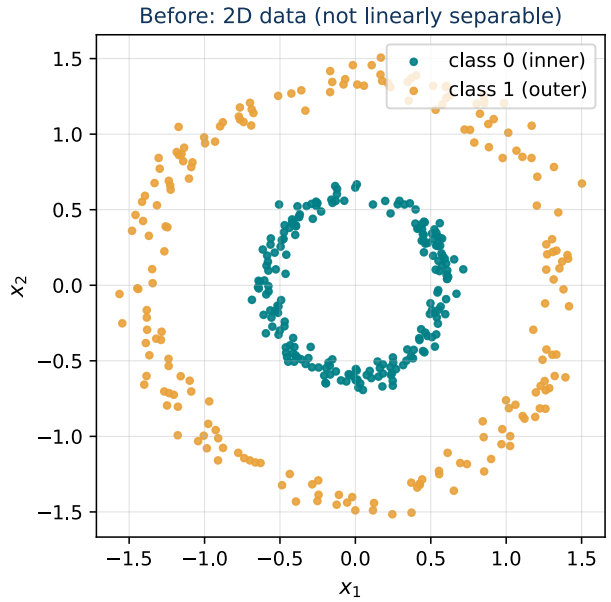
- Kernel PCA
- Isomap
- NMF — Nonnegative Matrix Factorization
- Factor Analysis
- Autoencoders

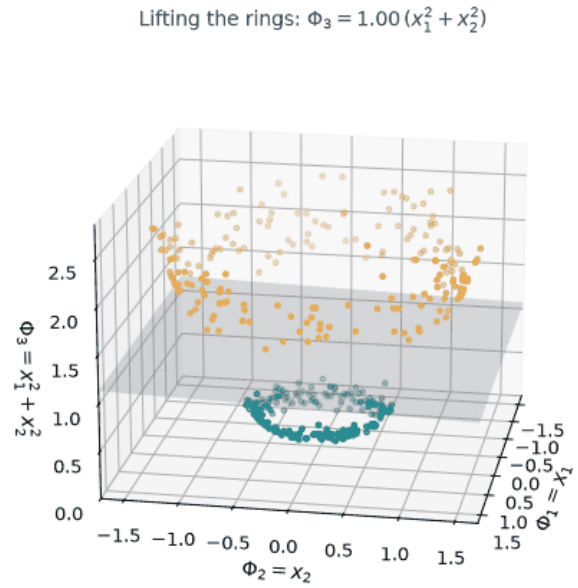
Short method summaries in the appendix. Next: the essential idea behind kernels.

Feature expansion and kernels

> Explore a richer feature space before finding a compact representation.

Feature Expansion (Higher Dimension)





The map in this animation: $\Phi_{\text{rings}}(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$ — add squared radius.

Other explicit expansions: polynomial features $\Phi(x_1, x_2) = (x_1,\ x_2,\ x_1^2,\ x_1 x_2,\ x_2^2,\ \ldots)$

Implicit expansion (kernels): use the kernel trick — inner product in a high-D space without computing $\Phi(\mathbf{x})$ explicitly.

Takeaway: Feature *expansion* increases dimensionality to make data more linearly separable (effectively you get a non-linear method from a linear one). Logistic regression on the rings: accuracy 0.55 in 2D, 1.00 after Φ_{rings}. · code example

Kernels: Compare Without Expanding

- **Explicit expansion:** construct new features, then take a dot product.
- **Kernel trick:** compute that dot product **without constructing the features**.

$$k(\mathbf{x}, \mathbf{z}) = \Phi(\mathbf{x})^\top \Phi(\mathbf{z})$$

A small example: $k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2$ corresponds to quadratic features. For many samples, collect pairwise kernel values in a **Gram matrix** K.

> **Fourth lecture:** kernel functions, the dual formulation, kernel regression and SVMs. Optional algebra and kernel PCA.

Kernel Trick example

A different quadratic map: two equivalent ways to compute its dot product.

1. Explicit feature expansion:

$$\Phi(\mathbf{x}) = (x_1^2,\ x_2^2,\ \sqrt{2}\, x_1 x_2)$$

$$\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j) = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2 x_{i1} x_{i2} x_{j1} x_{j2}$$

2. Direct computation (kernel trick):

$$(\mathbf{x}_i^\top \mathbf{x}_j)^2 = (x_{i1} x_{j1} + x_{i2} x_{j2})^2 = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2 x_{i1} x_{i2} x_{j1} x_{j2}$$

Conclusion: Computing $\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$ explicitly is the same as computing $(\mathbf{x}_i^\top \mathbf{x}_j)^2$. $\Rightarrow$ the kernel function avoids the explicit high-dimensional mapping.

Kernel PCA: PCA in a Nonlinear Feature Space

Conceptually: transform the inputs, then apply ordinary PCA there.

$$\underbrace{\mathbf{x}}_{d} \xrightarrow{\Phi} \underbrace{\Phi(\mathbf{x})}_{D} \xrightarrow{\text{linear PCA}} \underbrace{\mathbf{z}}_{k}.$$

With the kernel trick:

1. Compute pairwise inner products $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.
2. **Centre the Gram matrix** to centre the implicit feature space.
3. Use its leading eigenvectors and eigenvalues to obtain component scores.

Linear in the feature space, nonlinear in the original inputs. The kernel determines which structure PCA can capture.

Unsupervised; fitted kernel PCA can transform new samples. Detailed algebra · code example

Take-Home Messages

- **Selection:** keep original variables; filters, wrappers and embedded methods differ in how they choose.
- **PCA:** preserve variance with linear projections; choose scaling deliberately.
- **LDA:** use labels to find separating directions.
- **t-SNE / UMAP:** represent neighbourhoods; UMAP also transforms new samples before prediction.
- **Kernels:** implicit feature expansion — continued in the fourth lecture.

Recap Quiz

- Why can individually **relevant** features still be **redundant**?
- Why is trying every next addition **not exhaustive** subset search?
- What does **PCA optimize**, and what distinction can **LDA** retain that PCA loses?
- Which fitted methods can process **new samples**, and what does the **kernel trick** avoid computing?

Appendix: Three Search Strategies

Same task: choose a feature subset with the lowest validation loss.

Exhaustive / brute force

Evaluate **every** feasible subset.

Best score guaranteed.

Cost grows quickly.

Greedy

Take the **best next step**.

Commit to one path.

No global guarantee.

Random search

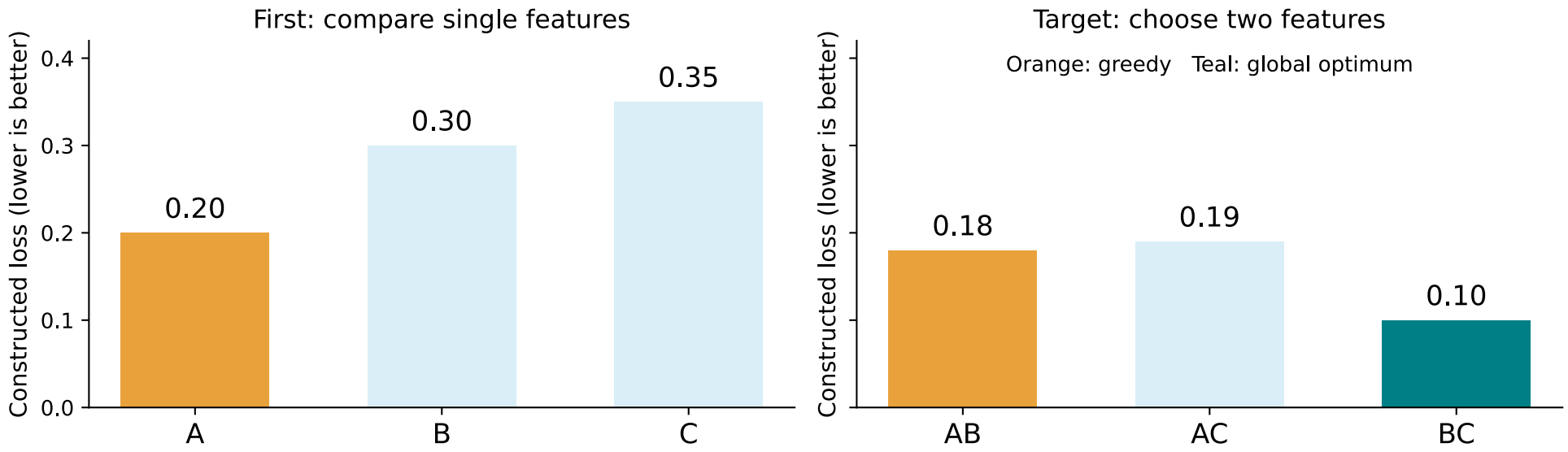
Sample **complete subsets** at random.

Set a trial budget.

No global guarantee.

Grid search is exhaustive search over a chosen parameter grid. Example: 3 values of C × 2 values of γ = **6 combinations**.

One Example: Greedy Can Miss the Best Pair



- **Greedy:** best single feature A → best pair containing it AB → loss **0.18**.
- **Brute force:** checks all three pairs → BC → loss **0.10**.
- **Random search:** can find BC if that pair is sampled.

Constructed example: choose exactly two features. Lower loss is better.

Heuristics: Practical Rules for Searching

A **heuristic** guides the search without a general global-optimality guarantee. **Greedy selection is one example.**

Local search

Start with a complete subset. **Swap one feature:** $AB \rightarrow BC$, replacing A with C. Loss **0.18 → 0.10**. Keep an improving move.

Random restarts

Repeat local search from different starting subsets. Keep the best result.

Small search space: brute force. **Limited budget:** compare greedy, random or local search under the same budget.

Appendix: LDA Details

Optional Fisher criterion and projection algebra.

LDA (Fisher) — Binary case in one slide

Goal: Find a direction $\mathbf{w}$ so that the two classes are *far apart after projection* $z = \mathbf{w}^\top \mathbf{x}$ but each class is *tight*.

After projection:

$$\tilde{\mu}_c = \mathbf{w}^\top \boldsymbol{\mu}_c \qquad \text{(projected class mean)}$$

$$\tilde{s}_c^{\,2} = \mathbf{w}^\top S_c\, \mathbf{w}, \quad S_c = \sum_{i:\, y_i = c} (\mathbf{x}_i - \boldsymbol{\mu}_c)(\mathbf{x}_i - \boldsymbol{\mu}_c)^\top \qquad \text{(projected within-class scatter)}$$

Scatter = sum of squared deviations: $\tilde{s}_c^2 = n_c \operatorname{Var}_c(z)$ (divisor n_c).

Fisher criterion (maximize separation / penalize spread):

$$J(\mathbf{w}) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{\tilde{s}_1^{\,2} + \tilde{s}_2^{\,2}} = \frac{\left(\mathbf{w}^\top(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)\right)^2}{\mathbf{w}^\top S_W \mathbf{w}}, \qquad S_W = S_1 + S_2.$$

Optimum: $\nabla J = 0$ gives $S_W \mathbf{w} \propto \boldsymbol{\mu}_1 - \boldsymbol{\mu}_2$ (the factors $\mathbf{w}^\top(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$ and $\mathbf{w}^\top S_W \mathbf{w}$ are scalars). For nonsingular S_W, one optimal direction (up to nonzero scale) is:

$$\boxed{\mathbf{w} = S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)}$$

For the multi-class case this leads to an eigenvalue problem: $S_W^{-1} S_B\, \mathbf{w} = \lambda\, \mathbf{w}$.

Appendix: t-SNE and UMAP Details

> From input distances to pairwise relationships, optimized coordinates and new-sample placement.

t-SNE: Turn Distances into Probabilities

Input relationships: $\mathbf{x}_i \in \mathbb{R}^d$; map $\mathbf{z}_i \in \mathbb{R}^m$, usually $m = 2$.

$$p_{j|i} = \frac{\exp\left(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / (2\sigma_i^2)\right)}{\sum_{\ell \neq i} \exp\left(-\|\mathbf{x}_i - \mathbf{x}_\ell\|^2 / (2\sigma_i^2)\right)}, \qquad p_{i|i} = 0.$$

- **Perplexity example:** four equally likely neighbours → 2 bits → perplexity 4.
- **Local scale:** choose each σ_i to match the requested perplexity.

$$\mathrm{Perp}(P_i) = 2^{H(P_i)}, \qquad H(P_i) = -\sum_{j \neq i} p_{j|i} \log_2 p_{j|i}.$$

Symmetrize: $p_{ij} = \dfrac{p_{j|i} + p_{i|j}}{2n}$, **probabilities sum to one:** $\sum_{i \neq j} p_{ij} = 1$.

t-SNE: Match Probabilities in the Map

Map relationships: a heavy-tailed similarity, normalized over pairs:

$$q_{ij} = \frac{(1 + \|\mathbf{z}_i - \mathbf{z}_j\|^2)^{-1}}{\sum_{a \neq b} (1 + \|\mathbf{z}_a - \mathbf{z}_b\|^2)^{-1}}, \qquad q_{ii} = 0.$$

Mismatch to minimize: optimize coordinates with P fixed:

$$\min_{\mathbf{z}_1, \ldots, \mathbf{z}_n} \mathcal{L} = \mathrm{KL}(P \| Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

- **Large p_{ij}, tiny q_{ij}:** input neighbours are separated too far.
- **Heavy tails:** moderate similarities can be represented at larger map distances.
- **Goal:** match neighbourhood probabilities, not original distances.

Original t-SNE paper

t-SNE: What Actually Moves the Points?

Update: the gradient of the exact objective above:

$$\nabla_{\mathbf{z}_i} \mathcal{L} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) \frac{\mathbf{z}_i - \mathbf{z}_j}{1 + \|\mathbf{z}_i - \mathbf{z}_j\|^2}.$$

A gradient step moves **against** this vector: $\mathbf{z}_i \leftarrow \mathbf{z}_i - \eta \nabla_{\mathbf{z}_i} \mathcal{L}$.

- $p_{ij} > q_{ij}$**:** this pair contributes attraction.
- $p_{ij} < q_{ij}$**:** this pair contributes repulsion.
- **Nonconvex:** initialization and optimization settings affect the layout.

> The point follows the **sum of all pair contributions**; no individual pair controls its motion.

UMAP: Build a Locally Scaled Graph

Input relationships: d_{ij} is the distance to a retained neighbour j:

$$a_{j|i} = \exp\left(-\frac{\max(0, d_{ij} - \rho_i)}{\sigma_i}\right).$$

- ρ_i**:** local distance offset; the nearest connections get full strength.
- σ_i**:** local scale, adjusted to the chosen neighbourhood size.
- **Outside the neighbour list:** set the directed weight to zero.

Combine directed strengths: fuzzy union; $0.8 + 0.5 - 0.8 \cdot 0.5 = 0.9$.

$$v_{ij} = a_{j|i} + a_{i|j} - a_{j|i}a_{i|j}.$$

$v_{ij} \in [0, 1]$ is an **edge membership strength**. Unlike t-SNE's p_{ij}, these weights do not sum to one.

UMAP: Fit a Low-Dimensional Graph

Map relationships: for output coordinates $\mathbf{z}_i$, define

$$q_{ij} = \frac{1}{1 + a\|\mathbf{z}_i - \mathbf{z}_j\|^{2b}}, \qquad a, b > 0.$$

Mismatch to minimize: graph cross-entropy (constant terms omitted):

$$\mathcal{L}_{\text{graph}} = -\sum_{i<j} \left[v_{ij} \log q_{ij} + (1 - v_{ij}) \log(1 - q_{ij}) \right].$$

- **Strong edges:** attraction favours larger q_{ij}, hence smaller distances.
- **Weak / absent edges:** repulsion discourages collapsing all points together.
- **`min_dist`, `spread`:** determine the curve parameters a, b.

Practical UMAP samples attractive edges and negative pairs; this changes their effective weighting. Paper · Sampling and the effective loss

UMAP: How New Samples Enter the Map

Keep the fitted training embedding Z_{train} fixed.

1. **Find neighbours** of $\mathbf{x}_{\text{new}}$ among training samples.
2. **Compute connection strengths** to those reference neighbours.
3. **Initialize** near a weighted combination of their map coordinates.
4. **Optimize the new coordinates** against the fixed reference embedding.

> The same fitted representation supports **training a classifier and later prediction**. Use `n_components` for the task; it need not be 2.

UMAP transform · scikit-learn t-SNE lacks `transform`; openTSNE offers placement in a reference map.

Appendix: Other Reduction Methods

> Different objectives preserve different aspects of the data.

Isomap: Follow Distances Along the Surface

1. Connect **nearby samples** in a weighted neighbour graph.
2. Approximate distances along the surface by **shortest paths**.
3. Find low-dimensional coordinates that reproduce those distances.

$$\|\mathbf{z}_i - \mathbf{z}_j\| \approx d_{\text{graph}}(i, j).$$

> On a rolled sheet, travel **along the sheet**, not straight across its folds.

NMF: Build Samples from Nonnegative Parts

For nonnegative data $X \in \mathbb{R}^{n \times d}$, choose $k < d$:

$$X \approx ZH, \qquad \min_{Z,H \geq 0} \|X - ZH\|_F^2.$$

- H: k component patterns, each with d features.
- Z: k nonnegative weights for each sample — the reduced representation.
- **Additive:** combine patterns without subtracting one from another.

Factor Analysis: Shared Factors and Separate Noise

Model each observation as

$$\mathbf{x} = \boldsymbol{\mu} + W\mathbf{z} + \boldsymbol{\epsilon}, \qquad \mathbf{z} \in \mathbb{R}^k, \quad k < d.$$

- $\mathbf{z}$: a few latent factors shared across features.
- W: loadings describing how each factor affects each feature.
- $\boldsymbol{\epsilon}$: feature-specific noise; each feature has its own variance.

> Explain **shared variation** separately from **feature-specific noise**.

Autoencoder: Learn to Compress and Reconstruct

$$\mathbf{x} \xrightarrow{\text{encoder } f_\theta} \mathbf{z} \in \mathbb{R}^k \xrightarrow{\text{decoder } g_\phi} \hat{\mathbf{x}}, \qquad k < d.$$

- **Encoder:** learns the reduced representation $\mathbf{z}$.
- **Decoder:** reconstructs the input from that representation.
- **Train both:** minimize reconstruction loss, for example

$$\min_{\theta,\phi} \frac{1}{n} \sum_i \|\mathbf{x}_i - g_\phi(f_\theta(\mathbf{x}_i))\|^2.$$

Appendix: Kernels and Kernel PCA

> Optional algebra; the fourth lecture develops kernels for prediction.

Kernel Trick by Hand: The Same Result

Use $\mathbf{x} = (1, 2)^\top, \mathbf{u} = (3, 4)^\top$ and

$$\Phi(\mathbf{x}) = (x_1^2,\ x_2^2,\ \sqrt{2}\, x_1 x_2)^\top.$$

Explicit transformation: 2D → 3D

$$\Phi(\mathbf{x}) = (1, 4, 2\sqrt{2})^\top$$
$$\Phi(\mathbf{u}) = (9, 16, 12\sqrt{2})^\top$$
$$\Phi(\mathbf{x})^\top \Phi(\mathbf{u}) = 9 + 64 + 48 = \boxed{121}.$$

Kernel: stay with the 2D inputs

$$k(\mathbf{x}, \mathbf{u}) = (\mathbf{x}^\top \mathbf{u})^2$$
$$= (1 \cdot 3 + 2 \cdot 4)^2$$
$$= 11^2 = \boxed{121}.$$

Same inner product, without constructing the expanded vectors. The factor $\sqrt{2}$ makes the cross term match.

PCA: Primal vs. Dual

Same PCA scores, from matrices of different sizes. X contains centred training rows.

Primal: a $d \times d$ matrix

$$S = \frac{1}{n} X^\top X, \qquad S\mathbf{w}_k = \lambda_k \mathbf{w}_k.$$

Project a centred new sample:

$$t_k = \mathbf{x}_{\text{new}}^\top \mathbf{w}_k.$$

Dual: an $n \times n$ matrix

$$S' = XX^\top, \qquad S'\boldsymbol{\alpha}_k = n\lambda_k \boldsymbol{\alpha}_k.$$

$$\mathbf{w}_k = \frac{X^\top \boldsymbol{\alpha}_k}{\sqrt{n\lambda_k}}, \qquad \lambda_k > 0.$$

$$t_k = \frac{\sum_i \alpha_{ik}\, \mathbf{x}_{\text{new}}^\top \mathbf{x}_i}{\sqrt{n\lambda_k}}.$$

Use **unit eigenvectors** and the **training mean** for centring. The smaller matrix depends on whether $d < n$ or $n < d$.

Kernel PCA

Fit: compute $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ and centre it:

$$\tilde{K} = HKH, \qquad H = I - \tfrac{1}{n}\mathbf{1}\mathbf{1}^\top, \qquad \tilde{K}\boldsymbol{\alpha}_k = n\lambda_k \boldsymbol{\alpha}_k.$$

Use unit eigenvectors $\boldsymbol{\alpha}_k$ with $\lambda_k > 0$; $\mathbf{1}$ is the vector of n ones.

Transform a new sample: first centre its kernel values using the training data:

$$\tilde{k}(\mathbf{x}, \mathbf{x}_i) = \underbrace{k(\mathbf{x}, \mathbf{x}_i)}_{\text{raw similarity}} - \underbrace{\frac{1}{n}\sum_j k(\mathbf{x}, \mathbf{x}_j)}_{\text{new-to-training mean}} - \underbrace{\frac{1}{n}\sum_j K_{ji}}_{\text{training column mean: stored}} + \underbrace{\frac{1}{n^2}\sum_{j,\ell} K_{j\ell}}_{\text{training grand mean: stored}}.$$

Then project:

$$t_k = \frac{1}{\sqrt{n\lambda_k}} \sum_i \alpha_{ik}\, \tilde{k}(\mathbf{x}, \mathbf{x}_i).$$

Transform uses the **fitted training reference**, without explicitly computing the feature space. code example